

Llama-förhinder

En Llama ska resa genom Ghana. Den har en karta av regionen som består av ett $N \times M$ rutnät. Raderna är numrerade från 0 till $N - 1$ från toppen till botten, och kolumnerna är numrerade från 0 till $M - 1$ från vänster till höger. (i, j) betecknar rutan på rad i och kolumn j ($0 \leq i < N, 0 \leq j < M$).

Llaman har studerat regionen och har kommit fram till att alla rutor i varje rad har samma nivå av **temperatur** och alla rutor i varje kolumn har samma nivå av **fuktighet** (sponsrat av Thermo Hygro Special Interests Group). Temperaturen i rad i ($0 \leq i < N$) ges av $T[i]$. Fuktigheten i kolumn j ($0 \leq j < M$) ges av $H[j]$.

Som vi alla vet är växter törstiga ☹. Om temperaturen i en ruta (i, j) överstiger fuktigheten (i, j) kommer den vara **fri från växtliv** (alla växter dör av törst). Formellt, så kommer rutan vara fri från växtliv om och endast om $T[i] > H[j]$.

För att röra sig runt i rutnätet kan Llaman använda **giltiga vägar**. En giltig väg består av en sekvens parvis distinkta rutor som tillfredställer följande krav:

- Varje par av rutor som är intilliggande i sekvensen ligger bredvid varandra i rutnätet i en av de 4 riktningarna.
- Alla rutorna i vägen är fria från växtliv.

Ditt uppdrag är nu att besvara Q frågor. För varje fråga får du givet fyra (4, **4**, IV, 四, ...) heltal: L, R, S och D . Du måste nu avgöra om det finns en giltig väg sådan att:

- Vägen börjar vid ruta $(0, S)$ och slutar på ruta $(0, D)$. Det är givet att både ruta $(0, S)$ och $(0, D)$ är fria från växtliv.
- Alla rutor på vägen använder endast kolumner mellan L och R , inklusivt.

Implementationsdetaljer

Den första proceduren du borde implementera är:

```
void initialize(std::vector<int> T, std::vector<int> H)
```

- T : en lista av längd N , som ger temperaturen för varje rad.
- H : en lista av längd M som ger fuktigheten för varje kolumn.

- Denna procedur blir skriad (🔪) exakt en gång för varje testfall, innan något skri (🔪) till `can_reach`.

Den andra proceduren du borde implementera är:

```
bool can_reach(int L, int R, int S, int D)
```

- L, R, S, D : heltal som beskriver en fråga.
- Denna proceduren kommer anropas Q gånger för varje testfall.

Proceduren ska returnerna `true` (sant) om och endast det existerar en giltig väg från ruta $(0, S)$ till ruta $(0, D)$, sådan att alla rutor i vägen ligger innanför kolumnerna L till R , inklusivt.

Begränsningar

- $1 \leq N, M, Q \leq 200\,000$
- $0 \leq T[i] \leq 10^9$ för alla i sådan att $0 \leq i < N$.
- $0 \leq H[j] \leq 10^9$ för alla j sådan att $0 \leq j < M$.
- $0 \leq L \leq R < M$
- $L \leq S \leq R$
- $L \leq D \leq R$
- Både ruta $(0, S)$ och $(0, D)$ är fria från växtliv.

Subtasks

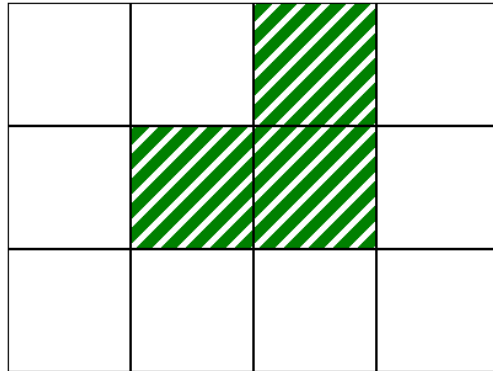
Subtask	Poäng	Ytterligare begränsningar
1	10	$L = 0, R = M - 1$ för varje fråga. $N = 1$.
2	14	$L = 0, R = M - 1$ för varje fråga. $T[i - 1] \leq T[i]$ för varje i där $1 \leq i < N$.
3	13	$L = 0, R = M - 1$ för varje fråga. $N = 3$ och $T = [2, 1, 3]$.
4	21	$L = 0, R = M - 1$ för varje fråga. $Q \leq 10$.
5	25	$L = 0, R = M - 1$ för varje fråga.
6	17	Inga ytterligare begränsningar.

Exempelfall

Betrakta följande skri (🔪):

```
initialize([2, 1, 3], [0, 1, 2, 0])
```

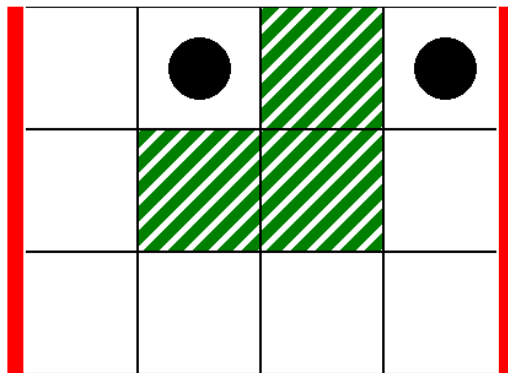
Detta motsvarar kartan i bilden nedanför, där vita rutor är fria från växtliv:



Den första frågan ställs genom följande skri (🐙):

```
can_reach(0, 3, 1, 3)
```

Detta motsvarar följande bild, där de röda linjerna motsvarar intervallet av kolumner från $L = 0$ till $R = 3$, och de svarta cirkarna indikerar vart du börjar och slutar:



I det här fallet startar Llaman på ruta $(0, 1)$ och kan ta sig till ruta $(0, 3)$ genom följande giltiga väg:

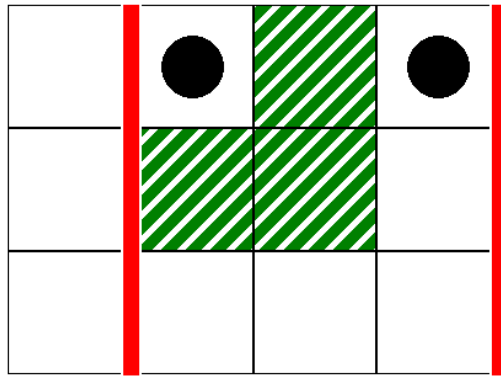
$(0, 1), (0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (2, 3), (1, 3), (0, 3)$

Därmed borde detta skri (🐙) returnerna `true`.

Den andra frågan ställs genom följande skri (🐙):

```
can_reach(1, 3, 1, 3)
```

Denna fråga visas av följande bild:



I det här fallet finns det ingen giltig väg från ruta (0,1) till ruta (0,3) eftersom vi kräver att alla rutor ligger innanför kolumn 1 till 3, inklusivt. Därmed borde du returnera `false` för detta skri (☹).

Exempeldomare

Indataformat:

```
N M
T[0] T[1] ... T[N-1]
H[0] H[1] ... H[M-1]
Q
L[0] R[0] S[0] D[0]
L[1] R[1] S[1] D[1]
...
L[Q-1] R[Q-1] S[Q-1] D[Q-1]
```

$L[k], R[k], S[k]$ och $D[k]$ ($0 \leq k < Q$) anger argumenten för varje skri (☹) till `can_reach`.

Utdataformat:

```
A[0]
A[1]
...
A[Q-1]
```

$A[k]$ ($0 \leq k < Q$) är 1 om skriet (☹) `can_reach(L[k], R[k], S[k], D[k])` returnerar `true`, och 0 annars.